Aula 06 — Aferição: medição e monitoração

Avaliação de Desempenho ()

Prof. Dr. Helvecio Bezerra Leal Neto

Universidade Federal do Oeste do Pará

15 de setembro de 2026

Roteiro

1. **O que é aferir?** A técnica e a carga de trabalho.
2. **Que carga usar?** Os critérios para escolhê-la.
3. **Quem registra o que aconteceu?** O monitor: evento, *overhead* e resolução.
4. **Que tipos de monitor existem?** E, na prática, aferir o seu script da Aula 05.

Meta

Sair sabendo planejar, no papel, **a carga** e **o monitor** de uma medição.

Na aula anterior: o log do SimPy

Saída do `fila_v1.py`:

```
0.0  c1 chega
0.0  c1 inicia (esperou 0.0 s)
2.0  c2 chega
3.0  c1 sai
3.0  c2 inicia (esperou 1.0 s)
5.0  c2 sai
6.0  c3 chega
6.0  c3 inicia (esperou 0.0 s)
8.0  c3 sai
```

Quem escreveu estas linhas?

O **próprio simulador**: ele sorteava (ou lia) os eventos e anotava cada um com um `print`.

A pergunta de hoje

E se o atendente for **de verdade**? Quem anota a chegada, o início e a saída?

Três técnicas, a mesma fila

A fila da Aula 05: um atendente, $\lambda = 0{,}8$ e $\mu = 1{,}0$ cliente/min. Qual é a utilização U?

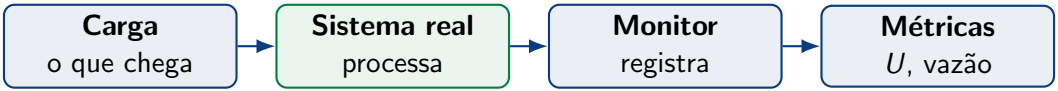
Técnica	**Aulas**	**Como obtém U**	**Resultado**
Modelagem	02 e 03	fórmula: $\rho = \lambda/\mu$	80 %
Simulação	04 e 05	`fila_v2.py`, semente 42	83,3 %
Aferição	**hoje**	medir o atendente real: ocupado ÷ total	**?**

Ideia-chave: a aferição é a única das três que olha o sistema de verdade.

O que é aferição

Definição

Aferição (medição ou experimentação) é monitorar um sistema computacional real, ou um protótipo dele, enquanto ele processa uma carga de trabalho específica.



Ideia-chave: aferir tem dois ingredientes: a carga que o sistema processa e o monitor que registra.

Quando aferir

- O sistema **já existe**, ou há um protótipo dele.
- A **precisão** do resultado é o que mais pesa na decisão.
- A pergunta é sobre **este** sistema, e não sobre um projeto.

O preço

O sistema precisa estar disponível para o experimento, e o próprio ato de medir pode perturbá-lo.

Exemplo: uma rede em produção precisa ser expandida, e a administração quer números confiáveis antes de autorizar o investimento.

Ideia-chave: afira quando o sistema existe e a pergunta exige o número dele, não o de um modelo.

Simular e aferir: o que muda

	Simulação (Aula 05)	**Aferição (hoje)**
Precisa do sistema?	não: imita até o que não existe	sim, ou um protótipo
Quem gera a carga?	o programa sorteia	usuários reais, ou um gerador
Que relógio conta?	`env.now`, o tempo simulado	o relógio real da máquina
Repetir dá igual?	sim, com a mesma semente	não exatamente
Observar custa?	não: o log não gasta tempo simulado	sim: o monitor usa o sistema

Ideia-chave: na aferição, o acaso vem do mundo real, e observar tem custo.

O que é carga de trabalho

Definição

Carga de trabalho (*workload*) é a lista de requisições que o sistema tem para atender.

Sistema	**Sua carga**
Rede de computadores	pacotes a transmitir
Banco de dados	consultas recebidas
Processador	instruções a executar
A fila da Aula 05	clientes: chegadas e serviços

Ideia-chave: a carga é a parte mais crucial do estudo: carga errada, conclusão errada.

Carga real ou carga sintética

Carga real

- Observada no sistema em **operação normal**.
- Fiel por definição.
- **Não se repete**: cada dia é diferente.

Carga sintética

- Um **modelo** da carga real.
- Aplicada **repetidamente**, de forma controlada.
- Gerada por um programa e injetada no sistema.

Na Aula 02 já vimos a diferença. Hoje ela decide **como** medir.

Ideia-chave: a carga real é fiel mas não se repete; a sintética se repete, se for representativa.

Você já usou uma carga sintética

O gerador de chegadas do `fila_v2.py` tem as qualidades de uma carga sintética:

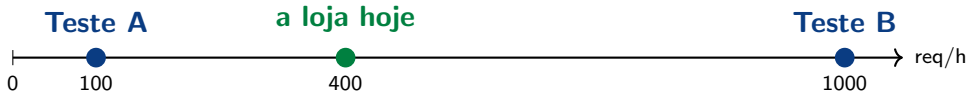
Qualidade	**No** `fila_v2.py`
Modelo da carga real	chegadas exponenciais com $\lambda = 0{,}8$
Repetível	`SEMENTE = 42`: mesma sequência (Teste 3)
Sem dados reais	nenhum cliente de verdade no arquivo
Fácil de modificar	os botões do topo: `LAMBDA`, `MU`...
Portável	pequena: roda no laboratório, no notebook e no Colab
Medição embutida	as listas `esperas`, `servicos` e `fila`

Ideia-chave: a semente é o que torna a carga sintética repetível; a carga real não tem semente.

Exemplo: a loja virtual (a situação)

A situação

Uma loja virtual recebe **400 requisições por hora**. O dono quer saber com que rapidez o servidor responde aos clientes **hoje**. Dois testes mediram esse tempo, cada um com uma carga:

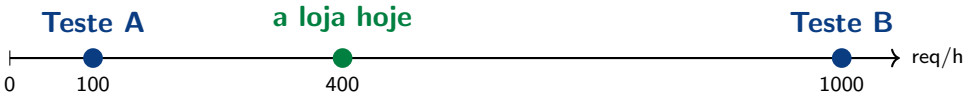


A pergunta

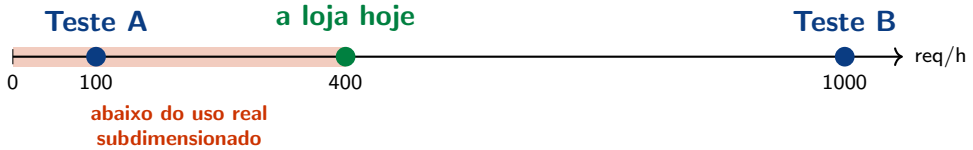
Algum dos dois testes dá ao dono o tempo de resposta da loja de hoje?

Fonte: Johnson & Coutinho (2011), seção 1.4.3.

Exemplo: a loja virtual (a resolução)

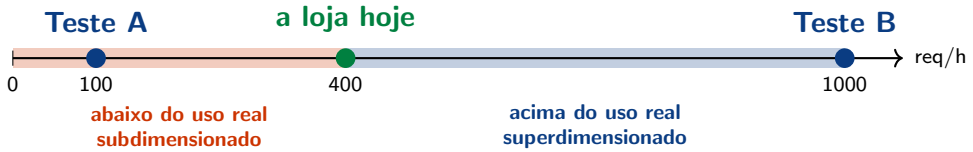


Exemplo: a loja virtual (a resolução)



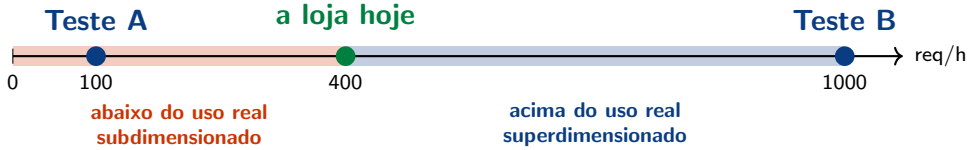
1. **Teste A:** só **um quarto** dos pedidos. Mediu resposta rápida, mas com 400 req/h o servidor fica sobrecarregado. **Engana.**

Exemplo: a loja virtual (a resolução)



1. **Teste A:** só **um quarto** dos pedidos. Mediu resposta rápida, mas com 400 req/h o servidor fica sobrecarregado. **Engana.**
2. **Teste B: 2,5 vezes** os pedidos. Mediu o tempo de uma loja maior: responde a **"e quando a loja crescer?"**

Exemplo: a loja virtual (a resolução)



1. **Teste A:** só **um quarto** dos pedidos. Mediu resposta rápida, mas com 400 req/h o servidor fica sobrecarregado. **Engana.**
2. **Teste B: 2,5 vezes** os pedidos. Mediu o tempo de uma loja maior: responde a **"e quando a loja crescer?"**
3. **Hoje:** só um teste com **400 req/h**. Nenhum dos dois fez isso.

Ideia-chave: abaixo do uso real, o teste engana; acima, ele fala do futuro.

Por que a carga leve engana tanto

Teste 1 da Aula 05: o atendente despacha **1 cliente por minuto**; só muda quantos chegam.

chegam por minuto	atendente ocupado	quem chega encontra, em média, esperando	espera média

Por que a carga leve engana tanto

Teste 1 da Aula 05: o atendente despacha **1 cliente por minuto**; só muda quantos chegam.

chegam por minuto	atendente ocupado	quem chega encontra, em média, esperando	espera média
0,5	51 %	●	1,06 min

Por que a carga leve engana tanto

Teste 1 da Aula 05: o atendente despacha **1 cliente por minuto**; só muda quantos chegam.

chegam por minuto	atendente ocupado	quem chega encontra, em média, esperando	espera média
0,5	51 %	●	1,06 min
0,8	83 %	● ● ● ● ●	5,91 min

Por que a carga leve engana tanto

Teste 1 da Aula 05: o atendente despacha **1 cliente por minuto**; só muda quantos chegam.

chegam por minuto	atendente ocupado	quem chega encontra, em média, esperando	espera média
0,5	51 %		1,06 min
0,8	83 %		5,91 min
0,95	94 %		**10,34 min**

Por que a carga leve engana tanto

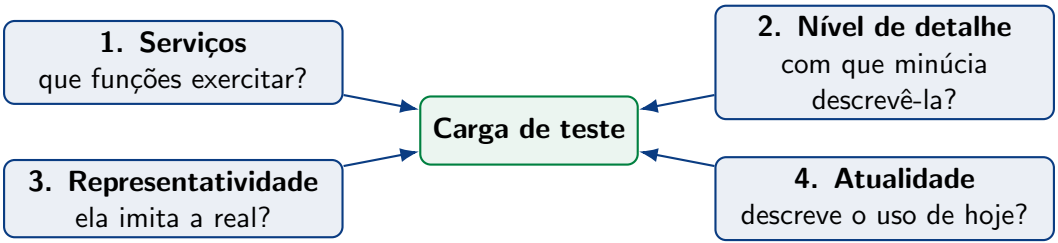
Teste 1 da Aula 05: o atendente despacha **1 cliente por minuto**; só muda quantos chegam.

chegam por minuto	atendente ocupado	quem chega encontra, em média, esperando	espera média
0,5	51 %	●	1,06 min
0,8	83 %	● ● ● ● ●	5,91 min
0,95	94 %	● ● ● ● ● ● ● ● ●	**10,34 min**

Cada pessoa na frente custa cerca de **1 min**, um atendimento. Quando a folga do atendente acaba, a fila dispara, e a espera junto.

Ideia-chave: a carga leve mede o sistema com folga: foi o que enganou o Teste A.

Quatro critérios para escolher a carga



Ideia-chave: antes de ligar o monitor, a carga passa por quatro perguntas.

Critério 1: serviços exercitados

Serviço é o que o sistema faz para quem o usa. A carga precisa **exercitar**, isto é, pôr para funcionar, os serviços que o estudo quer avaliar.

Critério 1: serviços exercitados

Serviço é o que o sistema faz para quem o usa. A carga precisa **exercitar**, isto é, pôr para funcionar, os serviços que o estudo quer avaliar.

Caso 1: um banco de dados

A carga do teste só tem consultas.

✓ consultar

× inserir × alterar

Falta serviço: inserir lento passa despercebido.

Critério 1: serviços exercitados

Serviço é o que o sistema faz para quem o usa. A carga precisa **exercitar**, isto é, pôr para funcionar, os serviços que o estudo quer avaliar.

Caso 1: um banco de dados

A carga do teste só tem consultas.

✓ consultar
× inserir × alterar

Falta serviço: inserir lento passa despercebido.

Caso 2: um programa de desenho

Dois são comparados só digitando texto.

✓ digitar texto
× desenhar, o que importa

Serviço errado: o que vence no texto pode ser o pior no desenho.

Critério 1: serviços exercitados

Serviço é o que o sistema faz para quem o usa. A carga precisa **exercitar**, isto é, pôr para funcionar, os serviços que o estudo quer avaliar.

Caso 1: um banco de dados

A carga do teste só tem consultas.

✓ consultar
× inserir × alterar

Falta serviço: inserir lento passa despercebido.

Caso 2: um programa de desenho

Dois são comparados só digitando texto.

✓ digitar texto
× desenhar, o que importa

Serviço errado: o que vence no texto pode ser o pior no desenho.

Ideia-chave: a carga exercita os serviços que o estudo quer avaliar, e todos eles.

Critério 2: nível de detalhe

Da nossa fila, vocês já usaram três descrições da mesma carga:

Critério 2: nível de detalhe

Da nossa fila, vocês já usaram três descrições da mesma carga:

A média
"cada um pede,
em média, 1 min"

modelagem, Aula 03

Critério 2: nível de detalhe

Da nossa fila, vocês já usaram três descrições da mesma carga:

A média
"cada um pede, em média, 1 min"

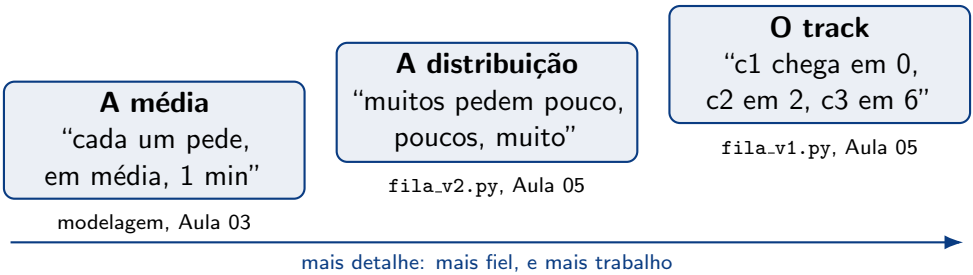
modelagem, Aula 03

A distribuição
"muitos pedem pouco, poucos, muito"

`fila_v2.py`, Aula 05

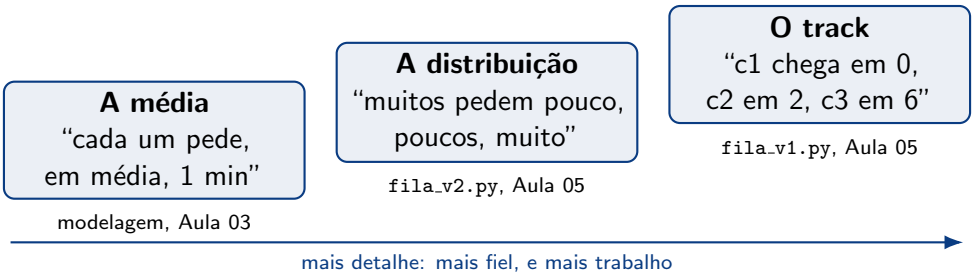
Critério 2: nível de detalhe

Da nossa fila, vocês já usaram três descrições da mesma carga:



Critério 2: nível de detalhe

Da nossa fila, vocês já usaram três descrições da mesma carga:



Com vários tipos de pedido, há ainda: o mais frequente, ou a frequência de cada tipo.

Ideia-chave: mais detalhe, mais fidelidade e mais trabalho: use o suficiente para a pergunta.

Critério 3: representatividade

A carga de teste deve **bater** com a real (ou ser proporcional) em três pontos:

Critério 3: representatividade

A carga de teste deve **bater** com a real (ou ser proporcional) em três pontos:

1. **Taxa de chegada:** quantos pedidos chegam. Na fila, λ; na loja, 400 req/h.
   Medida contando as chegadas no log. O Teste A errou aqui.

Critério 3: representatividade

A carga de teste deve **bater** com a real (ou ser proporcional) em três pontos:

1. **Taxa de chegada:** quantos pedidos chegam. Na fila, λ; na loja, 400 req/h.
   Medida contando as chegadas no log. O Teste A errou aqui.
2. **Demanda por recurso:** quanto trabalho cada pedido dá. Na fila, $1/\mu$.
   Medida cronometrando cada atendimento.

Critério 3: representatividade

A carga de teste deve **bater** com a real (ou ser proporcional) em três pontos:

1. **Taxa de chegada:** quantos pedidos chegam. Na fila, λ; na loja, 400 req/h.
   Medida contando as chegadas no log. O Teste A errou aqui.
2. **Demanda por recurso:** quanto trabalho cada pedido dá. Na fila, $1/\mu$.
   Medida cronometrando cada atendimento.
3. **Perfil de uso:** em que ordem e quanto CPU, disco e rede são usados.
   Medido observando os recursos enquanto o sistema trabalha.

Critério 3: representatividade

A carga de teste deve **bater** com a real (ou ser proporcional) em três pontos:

1. **Taxa de chegada:** quantos pedidos chegam. Na fila, λ; na loja, 400 req/h.
   Medida contando as chegadas no log. O Teste A errou aqui.
2. **Demanda por recurso:** quanto trabalho cada pedido dá. Na fila, $1/\mu$.
   Medida cronometrando cada atendimento.
3. **Perfil de uso:** em que ordem e quanto CPU, disco e rede são usados.
   Medido observando os recursos enquanto o sistema trabalha.

Ideia-chave: carga representativa tem a taxa, a demanda e o perfil da real, e quem os mede é um monitor.

Critério 4: atualidade

A pergunta

O uso que a carga imita ainda é o uso de **hoje**?

Os usuários se adaptam ao que o sistema faz bem. Exemplo:

Critério 4: atualidade

A pergunta

O uso que a carga imita ainda é o uso de **hoje**?

Os usuários se adaptam ao que o sistema faz bem. Exemplo:

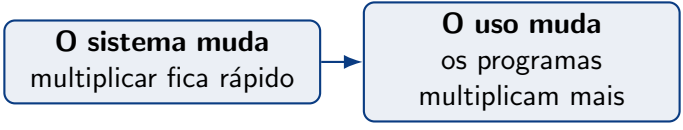
O sistema muda
multiplicar fica rápido

Critério 4: atualidade

A pergunta

O uso que a carga imita ainda é o uso de **hoje**?

Os usuários se adaptam ao que o sistema faz bem. Exemplo:

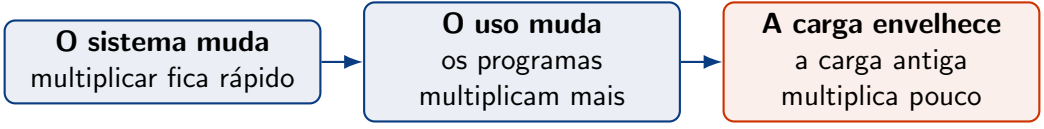


Critério 4: atualidade

A pergunta

O uso que a carga imita ainda é o uso de **hoje**?

Os usuários se adaptam ao que o sistema faz bem. Exemplo:

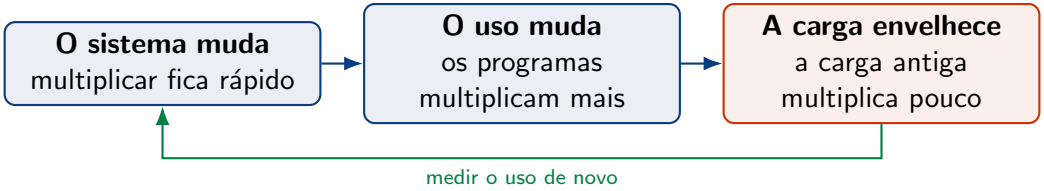


Critério 4: atualidade

A pergunta

O uso que a carga imita ainda é o uso de **hoje**?

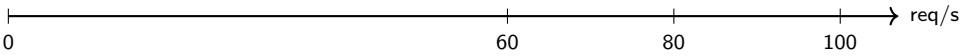
Os usuários se adaptam ao que o sistema faz bem. Exemplo:



Ideia-chave: a carga envelhece: monitore o uso continuamente e meça o de hoje, não o de ontem.

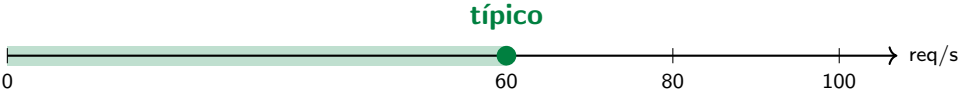
Nível de carga: típico, máximo ou além

O servidor de API: dois servidores de 40 req/s, **80 req/s** no total. Até onde carregá-lo no teste?



Nível de carga: típico, máximo ou além

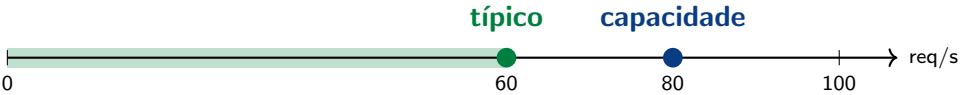
O servidor de API: dois servidores de 40 req/s, **80 req/s** no total. Até onde carregá-lo no teste?



1. **Típico (60 req/s, o uso real):** como ele se comporta no dia a dia?

Nível de carga: típico, máximo ou além

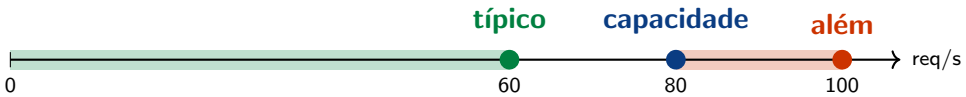
O servidor de API: dois servidores de 40 req/s, **80 req/s** no total. Até onde carregá-lo no teste?



1. **Típico (60 req/s, o uso real):** como ele se comporta no dia a dia?
2. **Capacidade (80 req/s):** até onde ele aguenta?

Nível de carga: típico, máximo ou além

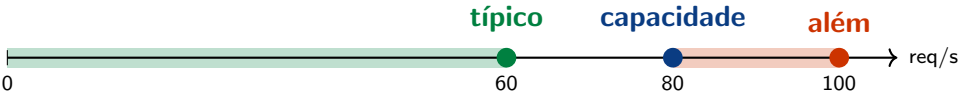
O servidor de API: dois servidores de 40 req/s, **80 req/s** no total. Até onde carregá-lo no teste?



1. **Típico (60 req/s, o uso real):** como ele se comporta no dia a dia?
2. **Capacidade (80 req/s):** até onde ele aguenta?
3. **Além (100 req/s):** o que acontece quando passa do limite? A vazão trava em 80 e o resto vira fila ou recusa.

Nível de carga: típico, máximo ou além

O servidor de API: dois servidores de 40 req/s, **80 req/s** no total. Até onde carregá-lo no teste?



1. **Típico (60 req/s, o uso real):** como ele se comporta no dia a dia?
2. **Capacidade (80 req/s):** até onde ele aguenta?
3. **Além (100 req/s):** o que acontece quando passa do limite? A vazão trava em 80 e o resto vira fila ou recusa.

Ideia-chave: como na loja, cada nível de carga responde a uma pergunta: declare qual é a sua.

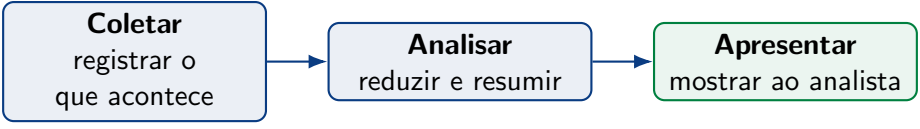
Intervalo

Pausa de 10 minutos

O que é um monitor

Definição

Monitor é a ferramenta que observa as atividades do sistema: coleta os dados, analisa e apresenta os resultados. Alguns também apontam problemas e sugerem soluções.



No simulador, era o Componente 3, a coleta de métricas. Agora ele vive dentro do sistema real.

Ideia-chave: a carga diz o que o sistema faz; o monitor registra o que aconteceu.

Você já escreveu um monitor

Trecho de `cliente` no `fila_v2.py`, com os comentários de hoje:

```
    with servidor.request() as req:
        fila.append(len(servidor.queue))    # COLETAR
        yield req
        w = env.now - chegada
        esperas.append(w)                   # COLETAR
        if log:                             # LIGA/DESLIGA
            print(f"{env.now:5.2f}  {nome} inicia (esperou {w:.2f})")   # APRESENTAR
        x = random.expovariate(mu)
        servicos.append(x)                  # COLETAR
```

No fim de `rodar()`, as médias fazem a **análise**. O `MOSTRAR_LOG` é o interruptor liga/desliga recomendado a todo monitor de software: monitorar custa.

Ideia-chave: as listas e o `print` da Aula 05 são um monitor de software.

Para que monitorar

Quem	**Para quê**
Programador	achar os trechos de software mais usados e otimizá-los
Gerente de sistemas	medir as utilizações e achar o **gargalo**
Gerente de sistemas	ajustar parâmetros do sistema (*tuning*)
Analista	caracterizar a carga, para planejar e criar cargas de teste
Analista	obter parâmetros de modelos e **validá-los**

Ideia-chave: monitorar é o primeiro passo de qualquer medição.

Exemplo: o monitor encontra o gargalo

Na Aula 02, um sistema foi observado por **60 s**. Quem limita o desempenho: a CPU ou o disco? Vamos refazer o trabalho do monitor.

Exemplo: o monitor encontra o gargalo

Na Aula 02, um sistema foi observado por **60 s**. Quem limita o desempenho: a CPU ou o disco? Vamos refazer o trabalho do monitor.

1. **Coletar**: somar o tempo em que cada recurso ficou ocupado. CPU: **57 s**; disco: **24 s**.

Exemplo: o monitor encontra o gargalo

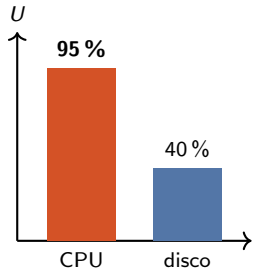
Na Aula 02, um sistema foi observado por **60 s**. Quem limita o desempenho: a CPU ou o disco? Vamos refazer o trabalho do monitor.

1. **Coletar**: somar o tempo em que cada recurso ficou ocupado. CPU: **57 s**; disco: **24 s**.
2. **Analisar**: $U = \text{ocupado} \div \text{total}$. CPU: $57/60 = 95\,\%$; disco: $24/60 = 40\,\%$.

Exemplo: o monitor encontra o gargalo

Na Aula 02, um sistema foi observado por **60 s**. Quem limita o desempenho: a CPU ou o disco? Vamos refazer o trabalho do monitor.

1. **Coletar**: somar o tempo em que cada recurso ficou ocupado. CPU: **57 s**; disco: **24 s**.
2. **Analisar**: U = ocupado $\div$ total. CPU: $57/60 = 95\,\%$; disco: $24/60 = 40\,\%$.
3. **Apresentar**: o gráfico mostra que a CPU é o **gargalo**, o recurso mais utilizado.



Ideia-chave: com o monitor, a decisão sai de números: trocar o disco não melhoraria nada.

Evento e traçado (trace)

Evento

Uma mudança no estado do sistema. Ex.: a chegada de um pacote. A mesma definição da Aula 04.

Traçado (*trace*)

O log dos eventos: instante, tipo e parâmetros de cada um.

O log da Aula 05 é um trace:

```
3.0  c2 inicia (esperou 1.0 s)
```

instante	`3.0`
tipo	`inicia`
parâmetros	c2, espera de `1.0 s`

Ideia-chave: a simulação escrevia o trace; a aferição precisa capturá-lo.

Overhead: observar também consome

Overhead (sobrecarga)

O consumo de CPU, memória e armazenamento causado pelo **próprio monitor**. Ele perturba o que mede.

No simulador

O `print` não avança `env.now`: com ou sem log, a espera média é 5,91 min.

No sistema real

O monitor roda na mesma máquina. Um medidor que acrescenta atraso à rede distorce o atraso medido.

A coleta "não intrusiva" da Aula 02 é isto: pouco *overhead*.

Ideia-chave: no simulador, observar é de graça; no sistema real, tem custo.

Exemplo: quanto custa observar (1/2)

A situação

A cada evento, o monitor de software "rouba" um pouco da CPU para anotá-lo: **0,1 ms por evento**. Chegam **100 eventos por segundo**. Quanto da CPU fica com o monitor?

0,1 ms = 100 instruções numa máquina de 1 milhão de instruções por segundo. Fonte: Jain (1991), seção 7.3.

Exemplo: quanto custa observar (1/2)

A situação

A cada evento, o monitor de software "rouba" um pouco da CPU para anotá-lo: **0,1 ms por evento**. Chegam **100 eventos por segundo**. Quanto da CPU fica com o monitor?

1. **Pense em um segundo como uma régua de 1 000 ms.**

1 segundo = 1 000 ms de CPU

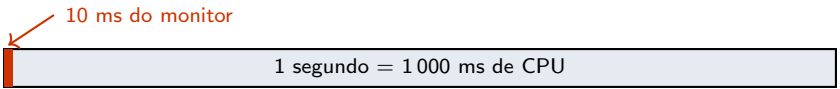
0,1 ms = 100 instruções numa máquina de 1 milhão de instruções por segundo. Fonte: Jain (1991), seção 7.3.

Exemplo: quanto custa observar (1/2)

A situação

A cada evento, o monitor de software "rouba" um pouco da CPU para anotá-lo: **0,1 ms por evento**. Chegam **100 eventos por segundo**. Quanto da CPU fica com o monitor?

1. **Pense em um segundo como uma régua de 1 000 ms.**
2. **Some o tempo do monitor nessa régua:** 100 eventos $\times$ 0,1 ms = **10 ms**.



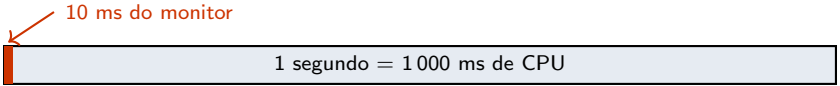
0,1 ms = 100 instruções numa máquina de 1 milhão de instruções por segundo. Fonte: Jain (1991), seção 7.3.

Exemplo: quanto custa observar (1/2)

A situação

A cada evento, o monitor de software "rouba" um pouco da CPU para anotá-lo: **0,1 ms por evento**. Chegam **100 eventos por segundo**. Quanto da CPU fica com o monitor?

1. **Pense em um segundo como uma régua de 1 000 ms.**
2. **Some o tempo do monitor nessa régua:** 100 eventos $\times$ 0,1 ms = **10 ms**.
3. **Divida pelo total:** $10 \div 1\,000 =$ **1 %** da CPU é do monitor.



0,1 ms = 100 instruções numa máquina de 1 milhão de instruções por segundo. Fonte: Jain (1991), seção 7.3.

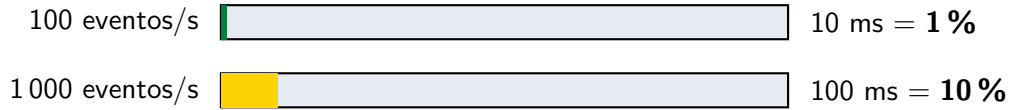
Exemplo: quanto custa observar (2/2)

E se os eventos forem mais frequentes? A mesma conta, com a mesma régua:

100 eventos/s  10 ms = **1 %**

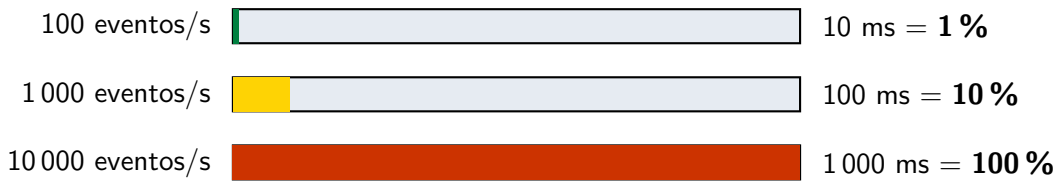
Exemplo: quanto custa observar (2/2)

E se os eventos forem mais frequentes? A mesma conta, com a mesma régua:



Exemplo: quanto custa observar (2/2)

E se os eventos forem mais frequentes? A mesma conta, com a mesma régua:



Com 10 000 eventos/s, o monitor ocuparia o segundo inteiro: não sobraria CPU para o sistema.

Ideia-chave: *overhead* é a utilização da CPU pelo monitor. Menos de 5 % costuma ser aceitável; eventos muito frequentes pedem outro tipo de monitor (Bloco 4).

Resolução: o menor detalhe visível

Resolução

A precisão da informação registrada: o menor intervalo que o monitor distingue.

Na Aula 05 (versão 2), c_2 esperou 0,29 min. E se o relógio só marcasse minutos inteiros?

Evento de c_2	**Instante real**	**Relógio de minutos inteiros**
chega	1,31	1
inicia	1,60	1
espera	**0,29**	**0**

Os dois instantes caem no mesmo minuto: o relógio escreve "1" para ambos.

Ideia-chave: abaixo da resolução, o que acontece não aparece.

Checagem do Bloco 3

1. Por que ligar o `MOSTRAR_LOG` não muda a espera média de 5,91 min?
2. Um monitor gasta 0,1 ms por evento. Com 500 eventos/s, qual é o *overhead*?
3. Um relógio que só marca minutos inteiros consegue medir uma espera de 20 segundos?

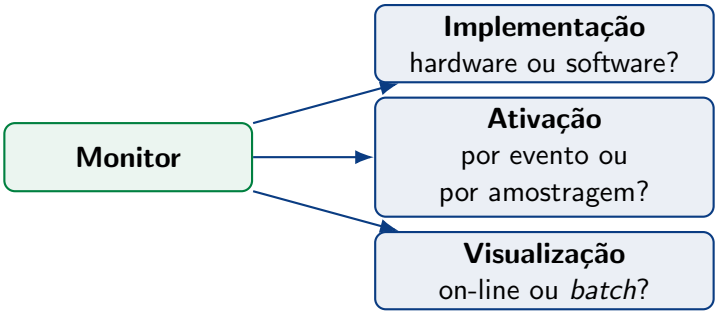
Checagem do Bloco 3

1. Por que ligar o `MOSTRAR_LOG` não muda a espera média de 5,91 min?
2. Um monitor gasta 0,1 ms por evento. Com 500 eventos/s, qual é o *overhead*?
3. Um relógio que só marca minutos inteiros consegue medir uma espera de 20 segundos?

Confira

1. O `print` não consome tempo simulado: no simulador, observar não perturba.
2. $500 \times 0{,}1 = 50$ ms em cada 1 000 ms: 5 %.
3. Não com precisão: conforme onde caem os instantes, ela aparece como 0 ou como 1 minuto.

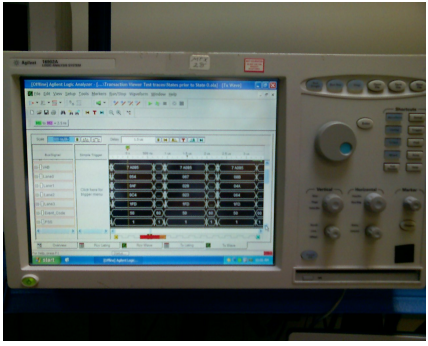
Três perguntas para classificar um monitor



Ideia-chave: as três perguntas são independentes: todo monitor tem uma resposta em cada uma.

Monitor de hardware: o observador silencioso

- Equipamento à parte, ligado ao sistema por **pontas de prova**.
- Não consome recursos do sistema medido: **sem overhead**.
- Relógio próprio: distingue **nanossegundos**.
- Caro, e exige conhecer bem o hardware.



Analisador lógico Agilent 16902A. Wikimedia Commons, domínio público.

Ideia-chave: o monitor de hardware mede de fora, por isso não pesa no sistema.

Hardware, software, firmware e híbrido

	Hardware	**Software**
Onde fica	fora, ligado por pontas de prova	dentro, rodando no próprio sistema
Resolução	até 10 ns	em geral 10 a 16 ms
Overhead	nenhum	depende da taxa de eventos
Custo	alto	médio

- **Firmware**: altera o microcódigo do processador; usado, por exemplo, em interfaces de rede.
- **Híbrido**: combina hardware, firmware e software.

Ideia-chave: o monitor de software é flexível e mais barato, mas paga em overhead e resolução.

Ativação: por evento ou por amostragem

Por evento

Acorda a cada evento de interesse. **Exato**. Bom para eventos raros; se frequentes, o *overhead* explode.

Por amostragem

Acorda em intervalos fixos. **Estima**. Bom para eventos frequentes: o custo não depende da taxa.

Por evento: registra quando algo muda



Por amostragem: olha a cada intervalo



Lembra a Aula 04? A câmera que só grava quando algo muda, e o brinquedo acionado a cada 10 minutos.

Ideia-chave: amostrar com período longo perde eventos; com período curto, custa mais.

Exemplo: medindo U de dois jeitos (a situação)

A fila das Aulas 04 e 05: c_1 ocupa o atendente de 0 a 3 s, c_2 de 3 a 5 s e c_3 de 6 a 8 s.



O que já sabemos (Aula 05)

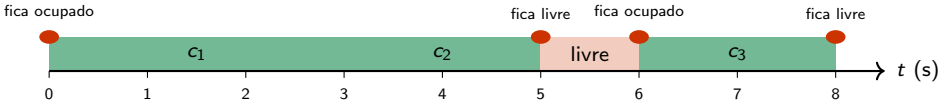
Ocupado $3 + 2 + 2 = 7$ s em 8 s: $U = 7/8 = \mathbf{87{,}5\,\%}$.

A pergunta

Um monitor **por evento** e um monitor **por amostragem** chegariam a esse número?

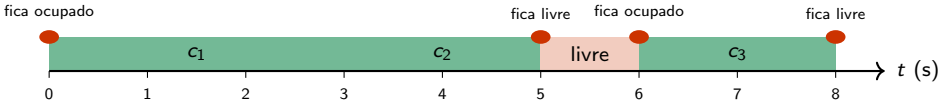
Exemplo: o monitor por evento

Ele só acorda quando o atendente **muda de estado**, e anota o instante:



Exemplo: o monitor por evento

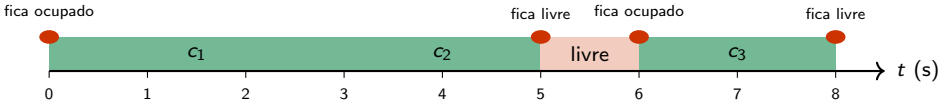
Ele só acorda quando o atendente **muda de estado**, e anota o instante:



1. **Anotações**: ocupado em 0, livre em 5, ocupado em 6, livre em 8.

Exemplo: o monitor por evento

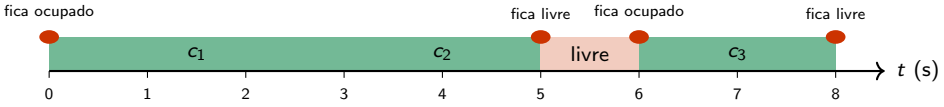
Ele só acorda quando o atendente **muda de estado**, e anota o instante:



1. **Anotações**: ocupado em 0, livre em 5, ocupado em 6, livre em 8.
2. **Tempo ocupado**: $(5 - 0) + (8 - 6) = 5 + 2 = 7$ s.

Exemplo: o monitor por evento

Ele só acorda quando o atendente **muda de estado**, e anota o instante:



1. **Anotações**: ocupado em 0, livre em 5, ocupado em 6, livre em 8.
2. **Tempo ocupado**: $(5-0)+(8-6)=5+2=7$ s.
3. **Utilização**: $7 \div 8 = \mathbf{87{,}5\,\%}$. Exato, com só 4 anotações.

Ideia-chave: por evento, o monitor vê cada mudança: o resultado é exato.

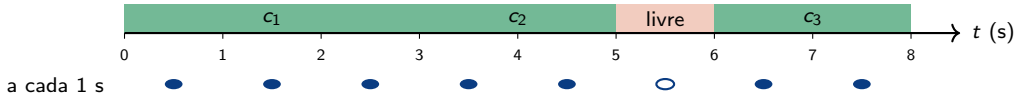
Exemplo: o monitor por amostragem

Ele olha o atendente em instantes fixos e anota "ocupado" (•) ou "livre" (○):



Exemplo: o monitor por amostragem

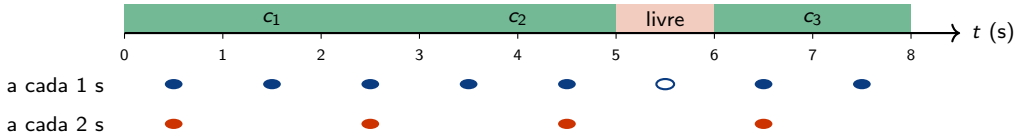
Ele olha o atendente em instantes fixos e anota "ocupado" (•) ou "livre" (○):



1. **A cada 1 s**: 7 das 8 olhadas acham "ocupado". $U = 7/8 = \mathbf{87{,}5\,\%}$. Acertou.

Exemplo: o monitor por amostragem

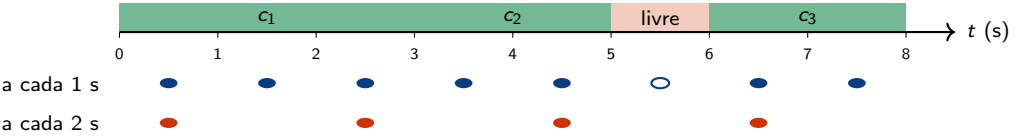
Ele olha o atendente em instantes fixos e anota "ocupado" (●) ou "livre" (○):



1. **A cada 1 s**: 7 das 8 olhadas acham "ocupado". $U = 7/8 = \mathbf{87{,}5\,\%}$. Acertou.
2. **A cada 2 s**: as 4 olhadas acham "ocupado". $U = 4/4 = \mathbf{100\,\%}$. Errou.

Exemplo: o monitor por amostragem

Ele olha o atendente em instantes fixos e anota "ocupado" (●) ou "livre" (○):



1. **A cada 1 s**: 7 das 8 olhadas acham "ocupado". $U = 7/8 = \mathbf{87{,}5\,\%}$. Acertou.
2. **A cada 2 s**: as 4 olhadas acham "ocupado". $U = 4/4 = \mathbf{100\,\%}$. Errou.
3. **Por quê?** O trecho livre (5 a 6 s) ficou **entre** as olhadas de 4,5 e de 6,5.

Ideia-chave: a amostragem estima: quanto maior o intervalo, mais coisa escapa.

Visualização: on-line ou batch

On-line

Analisa e mostra **durante** a coleta. Volumes pequenos. Ex.: acompanhar o tráfego de uma rede.

Batch

Coleta e guarda; analisa **depois**. Grandes volumes. Ex.: tempo médio de resposta, utilização de um processador.

No `fila_v2.py`: o log com `MOSTRAR_LOG = True` aparece durante a rodada (on-line); a espera média e o U saem das listas no fim (*batch*).

Ideia-chave: on-line serve para acompanhar; batch, para analisar muitos dados.

Checagem do Bloco 4: classifique

1. O monitor do `fila_v2.py`: hardware ou software? Por evento ou por amostragem? On-line ou batch?
2. "A cada 1 s, o monitor verifica se a rede está ocupada." Que ativação é essa?
3. Um analisador lógico gera *overhead* no sistema medido?

Checagem do Bloco 4: classifique

1. O monitor do `fila_v2.py`: hardware ou software? Por evento ou por amostragem? On-line ou batch?
2. "A cada 1 s, o monitor verifica se a rede está ocupada." Que ativação é essa?
3. Um analisador lógico gera *overhead* no sistema medido?

Confira

1. Software; por evento (anota a cada chegada e a cada início); log on-line e métricas *batch*.
2. Por amostragem.
3. Não: ele mede de fora, por pontas de prova.

Afira o seu script (20 min)

Até aqui o SimPy **imitava** um sistema. Agora o sistema é **real**: o seu computador rodando `simular()`. Numa célula nova do notebook da Aula 05:

```
import time
medidas = []
for log in (False, True):
    for rep in range(3):
        t0 = time.perf_counter()        # relogio REAL da maquina
        r = simular(mostrar_log=log)
        t1 = time.perf_counter()
        medidas.append((log, r["espera"], 1000 * (t1 - t0)))

for log, espera, ms in medidas:
    print(log, round(espera, 2), "min", round(ms, 2), "ms")
```

Carga sintética: 500 clientes com a sua matrícula. Monitor: `time.perf_counter()`, que lê o tempo real, e não o `env.now`.

O que observar e responder

Log	Espera (min)	Tempo real (ms)		
		rep. 1	rep. 2	rep. 3
desligado				
ligado				

1. A espera mudou com o log? Por quê?
2. E o tempo real? Que nome tem essa diferença?
3. As três repetições deram o mesmo tempo real? Compare com o Teste 3 da Aula 05.

Os números mudam de máquina para máquina. O que importa é o padrão.

O que o experimento mostra

1. **A espera não muda**: o `print` não avança `env.now`. O sistema simulado não sente o monitor.

O que o experimento mostra

1. **A espera não muda**: o `print` não avança `env.now`. O sistema simulado não sente o monitor.
2. **O tempo real, em geral, aumenta com o log**: imprimir consome a CPU da máquina que está sendo medida. É o *overhead*.

O que o experimento mostra

1. **A espera não muda**: o `print` não avança `env.now`. O sistema simulado não sente o monitor.
2. **O tempo real, em geral, aumenta com o log**: imprimir consome a CPU da máquina que está sendo medida. É o *overhead*.
3. **As repetições não dão o mesmo tempo real**, mesmo com a mesma semente: outros programas e o sistema operacional disputam a máquina. Na simulação, a mesma semente dava resultados idênticos.

Por isso uma medição precisa de repetições e de tratamento estatístico: é o assunto da Aula 07.

Ideia-chave: medir o sistema real traz o custo de observar e a variação entre medições.

O que você leva desta aula

1. **Aferir** é monitorar o sistema real sob uma carga específica.
2. A **carga** decide se o estudo vale: a real é fiel, a sintética é repetível.
3. Quatro critérios: serviços, nível de detalhe, representatividade e atualidade.
4. O **monitor** coleta, analisa e apresenta; acha gargalos e valida modelos.
5. **Evento**, **overhead** e **resolução**: o que observar, quanto custa e com que precisão.
6. Três eixos de classificação: implementação, ativação e visualização.

Para casa

Um monitor gasta 0,1 ms por evento. Quantos eventos por segundo cabem num *overhead* de 5 %? Leitura: Jain (1991), caps. 4, 5 e 7.

Próxima aula (Aula 07, 22/09)

- **Benchmarking**: cargas de teste padronizadas para comparar sistemas.
- **Ferramentas de medição** de rede e de sistema.
- **Projeto de experimentos**: que fatores variar, em que níveis, quantas repetições.
- **Credibilidade estatística**: médias e intervalos de confiança para a variação que vimos hoje e entre as sementes da Aula 05.
- **Síntese da Unidade I**: modelagem, simulação e aferição lado a lado.

Traga para a aula

As suas medições do script (a tabela com e sem log).

Referências



Jain, R. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation and Modeling*. John Wiley & Sons, 1991. Capítulos 4, 5 e 7.



Johnson, T. M. S. M.; Coutinho, M. M. *Avaliação de Desempenho de Sistemas Computacionais*. LTC, 2011. Seções 1.3 e 1.4.3; Capítulo 9.



Silva, M. *Avaliação de Desempenho de Sistemas: Técnicas de Aferição*. Notas de aula, UFOPA.



Python Software Foundation. *time: Time access and conversions* (`time.perf_counter`). `https://docs.python.org/3/library/time.html`